
Práctica Final Integradora

Informe Técnico

Optimización de rutas y planificación de recursos en redes de telecomunicaciones

Integrante 1: Nathalia Valentina Cardoza Azuaje

Integrante 2: Andres Felipe Velez Alvarez

Integrante 3: Sebastian Salazar Henao

Repositorio: ADA_PF_Cardoza_Salazar_Velez

8 de mayo de 2026

Índice

1. Introducción	3
2. Descripción del Dataset	3
3. Módulo A — Divide y Vencerás	3
3.1. MergeSort (orden descendente por <code>tenure</code>)	3
3.2. Búsqueda Binaria Recursiva	4
3.3. Resultados empíricos	4
3.4. Cumplimiento de la práctica (Módulo A)	5
3.5. Implementación por archivos (Módulo A)	5
4. Módulo B — Algoritmo Codicioso (Kruskal)	5
4.1. Algoritmo y Union-Find	5
4.2. Resultados	6
4.3. Cumplimiento de la práctica (Módulo B)	6
4.4. Implementación por archivos (Módulo B)	6
5. Módulo C — Programación Dinámica (Mochila 0-1)	6
5.1. Planteamiento y recurrencia	6
5.2. Resultados y contraejemplo codicioso	7
5.3. Cumplimiento de la práctica (Módulo C)	7
5.4. Implementación por archivos (Módulo C)	8
6. Integración del Pipeline	8
7. Conclusiones	8
8. Herramientas Utilizadas	9
9. Referencias	9
10. Roles del Equipo	9

1. Introducción

El proyecto aplica tres paradigmas algorítmicos al dataset *Telco Customer Churn* (Kaggle, 7043 clientes de una operadora en California) para gestionar solicitudes de servicio en una red de telecomunicaciones:

- **Módulo A — Divide y Vencerás:** Ordenar y consultar solicitudes por antigüedad. MergeSort garantiza $\Theta(n \log n)$ en el peor caso con estabilidad, y la búsqueda binaria recursiva localiza registros en $\Theta(\log n)$. Ningún algoritmo de ordenamiento por comparación puede superar estos límites asintóticos.
- **Módulo B — Codicioso (Kruskal):** Diseñar la red de mínimo costo entre 20 grupos de clientes. El problema del MST tiene *propiedad de elección codiciosa* (Lema del Ciclo), lo que permite que la decisión local óptima en cada paso produzca la solución global óptima.
- **Módulo C — Programación Dinámica:** Maximizar el ingreso del ancho de banda asignado a las 50 solicitudes más prioritarias. La Mochila 0-1 es NP-difícil y no admite solución codiciosa exacta, pero posee subestructura óptima y subproblemas solapados, las dos condiciones que hacen que la PD sea la técnica correcta.

2. Descripción del Dataset

El archivo `WA_Fn-UseC_-Telco-Customer-Churn.csv` tiene 7043 registros y 21 columnas. Se extraen cinco campos: `customerID` (identificador), `tenure` (prioridad en meses), `MonthlyCharges` (ingreso mensual), `TotalCharges` (peso en la mochila) y `Churn` (estado activo/en riesgo).

Estadísticas globales: 7043 registros totales; 11 con `TotalCharges` nulo (`tenure` = 0, se asigna 0.0); 5174 activos (`Churn` = No); `tenure` en $[0, 72]$ meses; `MonthlyCharges` promedio de 64.76 USD.

Manejo de nulos: `TotalCharges` aparece como cadena vacía cuando `tenure` = 0. El parser recorta espacios y caracteres `\r`, y asigna 0.0 a estos 11 registros, preservándolos en el dataset.

Construcción del grafo (Módulo B): Se forman 20 grupos asignando cada solicitud al grupo i mód 20. Para cada grupo se calcula el promedio de `MonthlyCharges` (redondeado a 2 decimales). El grafo completo de 20 nodos tiene $\binom{20}{2} = 190$ aristas con peso $\lfloor \overline{MC}_u + \overline{MC}_v \rfloor$, modelando el costo de enlace entre zonas de servicio.

3. Módulo A — Divide y Vencerás

3.1. MergeSort (orden descendente por tenure)

```

1 MergeSort(arr, izq, der):
2     si izq >= der: retornar
3     medio = (izq + der) / 2
4     MergeSort(arr, izq, medio); MergeSort(arr, medio+1, der)
5     Merge(arr, izq, medio, der) // fusiona en orden descendente
6
7 Merge(arr, izq, medio, der):
8     L = arr[izq..medio]; R = arr[medio+1..der]; k = izq
9     mientras i < |L| y j < |R|:
10         arr[k++] = (L[i].tenure >= R[j].tenure) ? L[i++] : R[j++]
11     copiar restos de L y R

```

Listing 1: MergeSort y Merge

Complejidad: $T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$ (Teorema Maestro, caso 2). Se elige sobre QuickSort por estabilidad y garantía $O(n \log n)$ en el peor caso.

Análisis ampliado (tiempo y espacio).

- **Tiempo (MergeSort):** la recurrencia $T(n) = 2T(n/2) + \Theta(n)$ surge de dividir en dos subarreglos y fusionar linealmente. El árbol de recursión tiene $\log_2 n$ niveles y costo $\Theta(n)$ por nivel.
- **Espacio (MergeSort):** la fusión requiere arreglos auxiliares temporales de tamaño total n , por lo que el uso adicional de memoria es $\Theta(n)$.
- **Costo total del Módulo A:** el ordenamiento domina asintóticamente:

$$T_A(n) = \Theta(n \log n) + q \cdot \Theta(\log n),$$

donde $q = 5$ consultas fijas; por tanto $T_A(n) = \Theta(n \log n)$.

3.2. Búsqueda Binaria Recursiva

```

1 BinSearch(arr, izq, der, k):
2     si izq > der: retornar -1
3     medio = (izq + der) / 2
4     si arr[medio].tenure == k: retornar medio
5     si arr[medio].tenure > k: retornar BinSearch(arr, medio+1, der, k)
6     retornar BinSearch(arr, izq, medio-1, k)

```

Listing 2: Búsqueda binaria (orden descendente)

Complejidad: $T(n) = T(n/2) + \Theta(1) \Rightarrow \Theta(\log n)$.

Análisis ampliado (búsqueda).

- **Tiempo:** en cada llamada se descarta la mitad del espacio de búsqueda, por lo que el número de niveles es acotado por $\lfloor \log_2 n \rfloor + 1$.
- **Espacio:** al ser recursiva, usa pila de llamadas de profundidad $\Theta(\log n)$.
- **Interpretación práctica:** para $n = 7043$, $\log_2 n \approx 12.8$, de modo que cada consulta requiere, en promedio, muy pocas comparaciones.

3.3. Resultados empíricos

n	MergeSort	Búsq. binaria (prom.)
1 000	136 μs	0.0110 μs
3 500	620 μs	0.0085 μs
7 043	1 209 μs	0.0075 μs

Cuadro 1: Tiempos empíricos (búsqueda: promedio de 10^6 ejecuciones).

Los tiempos de MergeSort son consistentes con $\Theta(n \log n)$: triplicar n incrementa el tiempo $\approx 4.6\times$ (teórico: $\approx 4.7\times$). Las 5 consultas fijas ($k \in \{72, 60, 45, 30, 12\}$) retornan correctamente los customerID 6904-JLBGY, 7426-WEIJX, 7925-PNRGI, 2684-EIWEO y 7450-NWRTR respectivamente. La salida se exporta a `results/solicitudes_ordenadas.csv` y alimenta al Módulo C.

Lectura cuantitativa adicional:

- Para $n = 1\,000$, $n \log_2 n \approx 9\,966$; para $n = 7\,043$, $n \log_2 n \approx 89\,975$. El cociente teórico de trabajo es ≈ 9.03 , cercano al cociente observado en tiempo ($1\,209/136 \approx 8.89$).
- La búsqueda binaria se promedió sobre 10^6 ejecuciones para disminuir ruido de reloj y sobrecarga de llamada, logrando medidas estables en escala submicrosegundo.

3.4. Cumplimiento de la práctica (Módulo A)

- **Parseo y nulos:** se cargan 7 043 registros y se normalizan 11 casos con `TotalCharges` vacío a 0.0, como exige la guía.
- **Ordenamiento requerido:** MergeSort estable en orden descendente por `tenure`, con justificación formal de $\Theta(n \log n)$ en peor caso.
- **Consultas obligatorias:** se ejecutan las 5 consultas fijas ($k = 72, 60, 45, 30, 12$) sobre el arreglo ordenado usando búsqueda binaria recursiva.
- **Análisis empírico:** se reportan tiempos para $n = 1\,000$, $n = 3\,500$ y $n = 7\,043$, verificando coherencia con la tendencia $\Theta(n \log n)$.

3.5. Implementación por archivos (Módulo A)

- `src/solicitud.hpp`: estructura `Solicitud`.
- `src/parser.cpp/.hpp`: lectura del CSV, limpieza de campos y manejo de nulos.
- `src/mergesort.cpp/.hpp`: ordenamiento estable por `tenure`.
- `src/binary_search.cpp/.hpp`: consulta recursiva en arreglo ordenado.
- `src/output.cpp/.hpp`: persistencia de salidas del módulo.
- `src/main.cpp`: orquestación de pruebas y medición de tiempos.

4. Módulo B — Algoritmo Codicioso (Kruskal)

4.1. Algoritmo y Union-Find

```

1 Kruskal(G):
2     ordenar aristas por peso ascendente
3     UF = UnionFind(G.numNodos)
4     para cada arista (u, v, w):
5         si NOT UF.conectados(u, v):
6             UF.unir(u, v); mst.agregar((u,v,w))
7             si |mst| == numNodos-1: romper
8     retornar mst
9
10 find(x): si padre[x]!=x: padre[x]=find(padre[x]); retornar padre[x]
11 unir(a,b): // union por rango; incrementa rango si empate

```

Listing 3: Kruskal con Union-Find por rango

Lema del Ciclo: Para cualquier ciclo C , la arista de mayor peso en C no pertenece a ningún MST. *Prueba:* si $e_{\text{máx}} \in T$ (MST), al eliminarla el árbol se biparticiona; existe $e' \in C$ con $w(e') < w(e_{\text{máx}})$ que cruza el corte, luego $T' = T - e_{\text{máx}} + e'$ tiene menor peso, contradicción. $\square$

Kruskal nunca incluye la arista de mayor peso de ningún ciclo, garantizando optimalidad global.

Complejidad: $O(E \log E)$ por el ordenamiento; Union-Find aporta $O(E \cdot \alpha(V)) \approx O(E)$.

Análisis del Módulo B.

- **Construcción del grafo:**
 - Agrupación de registros y acumulación de promedios: $\Theta(N)$.
 - Construcción de aristas completas entre 20 nodos: $\Theta(V^2)$ con $V = 20$, equivalente a 190 aristas.
- **Kruskal + Union-Find:**

- Ordenar aristas: $O(E \log E)$.
- Operaciones **find/union**: $O(E \cdot \alpha(V))$.
- Total: $O(E \log E)$ (el ordenamiento domina).
- **Espacio**: almacenamiento de aristas $O(E)$, estructura Union-Find $O(V)$ y MST resultante $O(V)$.

Con los tamaños de esta práctica ($V = 20$, $E = 190$), el costo real del módulo es muy bajo en tiempo absoluto, pero el análisis se presenta en forma general para mantener validez teórica.

4.2. Resultados

El MST tiene **19 aristas** y **peso total 2 388** (costo promedio de arista: 129.02). Las aristas van de peso 122 (nodos 1–12) a 130 (nodos 1–15), con topología estrellada en torno a los nodos 1 y 12, reflejo de que los grupos con cargas promedio más moderadas sirven de enlace más barato.

Verificación en subgrafo de 5 nodos $\{0, 1, 2, 12, 16\}$: Kruskal selecciona las aristas (1–12, 122), (12–16, 123), (0–1, 124), (2–12, 127) en ese orden, con peso total 496. Cualquier otro árbol de expansión del subgrafo tiene peso superior.

4.3. Cumplimiento de la práctica (Módulo B)

- **Construcción del grafo**: se reportan $V = 20$ nodos, $E = 190$ aristas y costo promedio de arista, tal como exige la práctica.
- **MST por Kruskal**: se lista el conjunto de 19 aristas del MST y su peso total.
- **Elección codiciosa**: se justifica formalmente con el Lema del Ciclo por qué seleccionar la menor arista que no forma ciclo mantiene optimalidad global.
- **Verificación manual**: se contrasta un subgrafo inducido de 5 nodos y se valida que coincide con la salida de Kruskal.

4.4. Implementación por archivos (Módulo B)

- **src/graph.cpp/.hpp**: agrupación determinista de clientes, cálculo de promedios por grupo y construcción de aristas del grafo completo.
- **src/kruskal.cpp/.hpp**: ordenamiento de aristas y ejecución de Union-Find con compresión de caminos y unión por rango.
- **src/main.cpp**: invocación del módulo, reporte de métricas de grafo y exportación de la solución MST a **results/mst_red.txt**.

5. Módulo C — Programación Dinámica (Mochila 0-1)

5.1. Planteamiento y recurrencia

Se toman las 50 solicitudes activas (**Churn** = No) de mayor **tenure** del arreglo ordenado por el Módulo A. Cada solicitud i se modela con $w_i = \text{round}(\text{TotalCharges})$ (peso) y $v_i = \text{round}(\text{MonthlyCharges} \times 10)$ (valor en centavos). Se busca maximizar $\sum_{i \in S} v_i$ sujeto a $\sum_{i \in S} w_i \leq W$.

$$dp[i][w] = \begin{cases} 0 & \text{si } i = 0 \text{ o } w = 0 \\ dp[i-1][w] & \text{si } w_i > w \\ \max\{dp[i-1][w], dp[i-1][w-w_i] + v_i\} & \text{si } w_i \leq w \end{cases}$$

El backtracking recupera los ítems seleccionados recorriendo la tabla de (n, W) hacia $(1, 0)$: si $dp[i][w] \neq dp[i-1][w]$, el ítem i fue incluido.

Justificación de escenarios: Con $W = 500$ ningún ítem del top-50 cabe (todos tienen $w_i > 500$, pues clientes de 72 meses acumulan miles en **TotalCharges**). Se evalúa también $W = 5\,000$ para observar el comportamiento real del algoritmo.

5.2. Resultados y contraejemplo codicioso

Enfoque	Solicitudes seleccionadas	Valor total	¿Óptimo?
$W = 500$ — PD	ninguna	0	Sí
$W = 5000$ — Codicioso (v/w)	4312-KFRXN, 6734-PSBAW, 7606-BPHHN	688	No
$W = 5000$ — PD (Mochila 0-1)	9286-BHDQG, 4312-KFRXN	707	Sí

Cuadro 2: Resultados de la Mochila y contraejemplo codicioso ($W = 500$ y $W = 5\,000$).

El codicioso selecciona 3 ítems (peso 4 904, valor 688) siguiendo el mayor ratio v/w . La PD descarta dos de ellos e incluye 9286-BHDQG (mayor valor absoluto pero menor ratio), logrando 707. Esto confirma que el criterio local óptimo puede bloquear combinaciones globalmente superiores cuando los pesos son heterogéneos.

Complejidad: $T(n, W) = \Theta(n \cdot W)$; espacio $\Theta(n \cdot W)$ (reducible a $\Theta(W)$ sin backtracking). El algoritmo es **pseudopolinomial**: polinomial en el valor numérico de W , pero exponencial en $\lceil \log_2 W \rceil$ bits.

Análisis del Módulo C.

- **Dimensión de tabla DP:** para $n = 50$, la tabla tiene $(n + 1) \times (W + 1) = 51 \times (W + 1)$ celdas.
 - Si $W = 500$: $51 \times 501 = 25\,551$ celdas.
 - Si $W = 5\,000$: $51 \times 5001 = 255\,051$ celdas.
- **Tiempo de tabulación:** cada celda se resuelve en $O(1)$, por lo que el costo total es $\Theta(nW)$.
- **Backtracking:** recorre a lo sumo n filas, costo adicional $O(n)$, dominado por $\Theta(nW)$.
- **Espacio:** tabla completa $\Theta(nW)$; versión optimizada sin reconstrucción puede reducirse a $\Theta(W)$.
- **Pseudopolinomialidad formal:** si la capacidad se codifica en $b = \lceil \log_2 W \rceil$ bits, entonces W puede crecer como 2^b , y $\Theta(nW) = \Theta(n2^b)$ no es polinomial en el tamaño binario de la entrada.

Escala real de memoria: para $n = 50$ y $W = 5\,000$, la tabla tiene $(n + 1)(W + 1) = 51 \times 5001 = 255\,051$ celdas. Con enteros de 4 bytes, el consumo aproximado es cercano a 1.0 MB.

5.3. Cumplimiento de la práctica (Módulo C)

- **Tabulación 0-1:** se construye la tabla de $(n + 1) \times (W + 1)$ con $n = 50$, incluyendo los escenarios $W = 500$ y $W = 5\,000$.
- **Contraejemplo codicioso:** se documenta un caso explícito con exactamente 3 solicitudes donde el criterio por ratio v/w no alcanza el óptimo.

- **Backtracking:** se reconstruye el conjunto final de solicitudes a partir de la tabla DP para reportar IDs seleccionados.
- **Complejidad formal:** se justifica $\Theta(nW)$ en tiempo y espacio y se discute la pseudopolinomialidad respecto al tamaño binario de W .

5.4. Implementación por archivos (Módulo C)

- `src/knapsack.cpp/.hpp`: modelado de ítems, tabulación DP, backtracking y construcción del contraejemplo codicioso.
- `src/main.cpp`: ejecución de escenarios $W = 500$ y $W = 5\,000$.
- `results/asignacion_bw_500.txt` y `results/asignacion_bw_5000.txt`: evidencia del resultado final.

6. Integración del Pipeline

El programa (`main.cpp`) ejecuta los módulos en el orden $B \rightarrow A \rightarrow C$, aunque conceptualmente el pipeline lógico es $A \rightarrow B \rightarrow C$:

1. **Carga:** `parser.cpp` lee el CSV, limpia nulos y construye el vector `solicitudes` (7043 elementos).
2. **Módulo B:** `graph.cpp` agrupa las solicitudes y construye el grafo; `kruskal.cpp` calcula el MST y lo exporta a `results/mst_red.txt`.
3. **Módulo A:** `mergesort.cpp` ordena el vector *in place* por *tenure* descendente; `binary_search.cpp` ejecuta las 5 consultas. Salidas: `solicitudes_ordenadas.csv` y `busquedas_A.txt`.
4. **Módulo C:** `knapsack.cpp` toma el mismo vector ya ordenado (sin re-leer el CSV), selecciona el top-50 activo y ejecuta la PD para $W = 500$ y $W = 5\,000$. Salidas: `asignacion_bw_500.txt` y `asignacion_bw_5000.txt`.

La integración $A \rightarrow C$ es directa en memoria: el arreglo ordenado por MergeSort es consumido inmediatamente por `construirItemsMochila()`, garantizando que los 50 candidatos sean los clientes activos de mayor antigüedad.

Como verificación de extremo a extremo, una corrida completa debe generar cinco artefactos en `results/`: `solicitudes_ordenadas.csv`, `busquedas_A.txt`, `mst_red.txt`, `asignacion_bw_500.txt` y `asignacion_bw_5000.txt`.

7. Conclusiones

Paradigma	Algoritmo	Complejidad	Garantía de optimalidad
Divide y Vencerás	MergeSort + Bin. Search	$\Theta(n \log n)$ / $\Theta(\log n)$	Exacto: solución óptima para ordenamiento y búsqueda por comparación.
Codicioso	Kruskal (MST)	$O(E \log E)$	Exacto para MST: Lema del Ciclo garantiza que la elección local produce el óptimo global.
Prog. Dinámica	Mochila 0-1	$\Theta(n \cdot W)$	Exacto pero pseudopolinomial; no existe algoritmo polinomial salvo $P=NP$.

Cuadro 3: Comparación de paradigmas.

- **Datos reales vs. sintéticos:** El escenario $W = 500$ con valor óptimo 0 ilustra cómo la escala real del dataset puede hacer que el algoritmo produzca resultados triviales que no

aparecerían con instancias sintéticas calibradas. La diferencia de 19 centavos (PD vs. codicioso con $W = 5\,000$) es pequeña en valor absoluto pero confirma que la estructura real de los pesos no satisface la propiedad de elección codiciosa.

- **Codicioso exacto vs. heurístico:** Kruskal es codicioso exacto (la elección local es suficiente para la optimalidad global). El codicioso por ratio v/w en la Mochila es solo una heurística sin garantía de optimalidad.
- **Pipeline:** La arquitectura en memoria (sin re-lectura del CSV entre módulos) demuestra la importancia de diseñar estructuras de datos compartidas que eviten trabajo redundante.

8. Herramientas Utilizadas

C++17 (`g++ -std=c++17 -O2`), Visual Studio Code, Git/GitHub, L^AT_EX (pdf_latex, paquetes `babel`, `booktabs`, `listings`). Se utilizó **Claude** (Anthropic, Sonnet 4.6) como asistente para redacción, revisión de pseudocódigos y análisis de complejidad del informe. El código fuente del programa fue escrito íntegramente por el equipo. Uso declarado conforme a las políticas del curso.

9. Referencias

- [1] Blastchar. *Telco Customer Churn*. Kaggle, 2018. <https://www.kaggle.com/datasets/blastchar/telco-customer-churn>
- [2] Cormen, T.H. et al. (2022). *Introduction to Algorithms* (4.^a ed.). MIT Press.
- [3] Material del curso ADA — Universidad EAFIT, 2026-1.
- [4] Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Pearson.

10. Roles del Equipo

Integrante	Contribuciones específicas
Nathalia Valentina Cardoza Azuaje	Módulo A. <code>mergesort.cpp</code> , <code>binary_search.cpp</code> , <code>parser.cpp</code> , <code>output.cpp</code> : parseo del CSV, manejo de nulos, MergeSort estable, búsqueda binaria recursiva, medición de tiempos. Redacción de las secciones del Módulo A.
Sebastian Salazar Henao	Módulo B. <code>graph.cpp</code> , <code>graph.hpp</code> , <code>kruskal.cpp</code> , <code>kruskal.hpp</code> : construcción del grafo, Union-Find por rango, MST, <code>guardarMST()</code> . Integración final en <code>main.cpp</code> , coordinación del repositorio Git. Redacción Módulo B.
Andres Felipe Velez Alvarez	Módulo C. <code>knapsack.cpp</code> , <code>knapsack.hpp</code> : tabulación PD, backtracking, contraejemplo codicioso, reportes <code>asignacion_bw_*.txt</code> . Redacción de las secciones del Módulo C y revisión de la integración A→C.

Cuadro 4: Distribución de roles por módulo.